

Course Outline: Measuring Frontend Performance (Core Web Vitals)

Title: Measuring Frontend Performance (Core Web Vitals) **Learnpack:** + Midiendo el desempeño del frontend (Core Web Vitals)
Prerequisite: Debugging with DevTools (same day, previous learnpack) **Target Audience:** Beginner frontend developers learning to evaluate application performance **Total Lessons:** 9 **Estimated Total Length:** ~4,500 words **Format:** Conceptual (0% hands-on)

Course Description

This course introduces students to the critical practice of measuring frontend performance. Before you can optimize anything, you need to understand what to measure and why it matters. Students will learn the Core Web Vitals metrics that Google uses to evaluate user experience: First Contentful Paint, Largest Contentful Paint, and Total Blocking Time.

The course emphasizes a measurement-first mindset. Students will understand why "you can't improve what you don't measure" and how performance directly impacts business outcomes. By the end, students will be able to read a Lighthouse report, identify which metrics need attention, and understand what each metric tells them about their application's user experience.

This learnpack focuses purely on understanding metrics—the next learnpack covers optimization strategies like code splitting, lazy loading, and memoization.

Course Philosophy

- This course emphasizes:
- Measurement before optimization (understand the problem before solving it)
 - User-centric metrics (what the user experiences, not just technical numbers)
 - Practical interpretation (reading real reports, not memorizing definitions)

Course Statistics Summary

Section	Lessons	Estimated Words
00 - Welcome	1	~400
01 - Why Measurement Matters	2	~1,000
02 - Core Web Vitals Explained	3	~1,500
03 - Reading Performance Reports	1	~500
04 - Assessment	1	~650
05 - Conclusion	1	~400
Total	9	~4,450

Section 00: Welcome

00.0 Welcome to Frontend Performance Measurement 📖 (~400 words)

Learning Objectives:

- Understand what this course covers and why it matters
- Recognize the connection between performance measurement and user experience

Content Outline:

- Course introduction: what students will learn
- The fundamental principle: you can't improve what you don't measure
- Brief overview of Core Web Vitals as industry-standard metrics
- How this course fits with the debugging learnpack (just completed) and optimization learnpack (coming next)
- What students will be able to do after completing this course

Transitions: Opens the course by framing performance measurement as an essential skill. Connects to the debugging tools students just learned (DevTools) and prepares them for optimization strategies they'll learn next.

Key Principles:

- Performance is about user experience, not technical vanity
- Measurement provides evidence for decision-making
- Industry-standard metrics help communicate across teams

Examples:

- A website that "feels slow" vs. having data showing LCP is 4.2 seconds
 - How a product manager and developer can discuss performance using shared vocabulary (Core Web Vitals)
-

Section 01: Why Measurement Matters

01.0 You Can't Improve What You Don't Measure 📖 (~500 words)

Learning Objectives:

- Explain why measurement must precede optimization
- Identify the risks of optimizing without baseline data

Content Outline:

- The measurement-first mindset: why guessing doesn't work
- Common mistakes: optimizing the wrong thing, premature optimization
- How measurement provides direction and priority
- The feedback loop: measure → change → measure again
- Real numbers vs. subjective feelings ("it feels faster")

Transitions: Establishes the philosophical foundation for the course. The next lesson connects this principle to real business outcomes.

Key Principles:

- Evidence over assumption (connects to DevTools lesson from previous learnpack)
- Optimization without measurement is guessing
- Numbers create accountability and clarity

Examples:

- Developer spends 3 days optimizing image loading, but the real bottleneck was JavaScript execution
 - Team argues about whether the site is "fast enough" — measurement ends the debate
 - Before/after comparison only possible with baseline measurements
-

01.1 The Business Impact of Performance 📖 (~500 words)

Learning Objectives:

- Connect frontend performance to business metrics
- Explain why companies invest in performance optimization

Content Outline:

- Performance affects user behavior: bounce rates, conversions, engagement
- Industry research: the cost of slow pages (Amazon, Google, Walmart studies)
- Mobile users and emerging markets: performance as accessibility
- SEO implications: Core Web Vitals as Google ranking factor
- Performance budgets: treating speed as a feature requirement

Transitions: Moves from "why measure" to "what to measure." Students now understand the stakes; the next section introduces the specific metrics.

Key Principles:

- Performance is a business metric, not just a technical one
- Every second matters: small delays have measurable impact
- Good performance is inclusive (works on slower devices/connections)

Examples:

- Pinterest reduced perceived wait time by 40%, signups increased 15%
 - Google found 53% of mobile users abandon sites taking over 3 seconds
 - A slow checkout page directly costs revenue
-

Section 02: Core Web Vitals Explained

02.0 What Are Core Web Vitals? 📖 (~500 words)

Learning Objectives:

- Define Core Web Vitals and their purpose
- Identify the three main metrics and what each measures

Content Outline:

- Core Web Vitals: Google's initiative to quantify user experience
- The three pillars: loading, interactivity, visual stability
- Overview of each metric (detailed in following lessons):
 - Largest Contentful Paint (LCP): loading performance
 - First Input Delay (FID) / Total Blocking Time (TBT): interactivity
 - Cumulative Layout Shift (CLS): visual stability (brief mention)
- Why these specific metrics were chosen: user-centric, measurable, actionable
- Good, Needs Improvement, Poor thresholds

Transitions: Introduces the framework. Following lessons dive deep into each metric students need to understand.

Key Principles:

- Core Web Vitals measure what users actually experience
- Three dimensions: loading, interactivity, stability
- Standardized thresholds enable comparison and goal-setting

Examples:

- A page might load data quickly but feel frozen (good LCP, bad TBT)

- Content jumping around as ads load (CLS problem)
 - The difference between technical "page load" and user-perceived "ready to use"
-

02.1 First Contentful Paint and Largest Contentful Paint 📖 (~500 words)

Learning Objectives:

- Differentiate between FCP and LCP
- Identify what triggers each metric and their target thresholds

Content Outline:

- First Contentful Paint (FCP): when the first content appears
 - What counts as "content" (text, images, canvas)
 - Why it matters: user perception that "something is happening"
 - Target threshold: under 1.8 seconds
- Largest Contentful Paint (LCP): when the main content is visible
 - What counts as "largest" (images, text blocks, video posters)
 - Why it matters: user perception that "the page is useful"
 - Target threshold: under 2.5 seconds
- The relationship between FCP and LCP
- Common elements that become the LCP element

Transitions: Covers the "loading" dimension of Core Web Vitals. Next lesson addresses interactivity.

Key Principles:

- FCP = "something is happening"; LCP = "I can use this page"
- LCP is more important than FCP for user experience
- The LCP element is often a hero image or heading

Examples:

- A page showing a loading spinner (FCP) but no real content for 4 seconds (bad LCP)
 - E-commerce product page: LCP is usually the product image
 - Blog post: LCP is often the headline or featured image
-

02.2 Total Blocking Time and Interactivity 📖 (~500 words)

Learning Objectives:

- Explain what Total Blocking Time measures
- Connect TBT to user-perceived interactivity

Content Outline:

- The interactivity problem: page looks ready but doesn't respond
- First Input Delay (FID): field metric measuring real user interaction delay
- Total Blocking Time (TBT): lab metric measuring potential for delay
 - How TBT is calculated: sum of blocking portions of long tasks
 - What makes a task "long" (over 50ms)
 - Target threshold: under 200 milliseconds
- Why TBT is used in tools like Lighthouse (lab vs. field metrics)
- The main thread and why blocking it ruins user experience
- JavaScript as the primary cause of blocking time

Transitions: Completes the Core Web Vitals deep dive. Students now understand what each metric measures. Next section teaches them how to read this data in real reports.

Key Principles:

- Interactivity matters: users expect immediate response to clicks/taps
- Long JavaScript tasks block the main thread
- TBT predicts how frustrated users will be trying to interact

Examples:

- User clicks a button, nothing happens for 300ms (feels broken)
 - Heavy JavaScript framework initialization blocking all interaction
 - The difference between a page that "looks done" and one that "is done"
-

Section 03: Reading Performance Reports

03.0 Understanding Lighthouse and Performance Scores 📖 (~500 words)

Learning Objectives:

- Navigate a Lighthouse performance report
- Interpret the overall score and individual metrics
- Distinguish between lab data and field data

Content Outline:

- Lighthouse: Chrome's built-in performance auditing tool
- How to run a Lighthouse audit (DevTools → Lighthouse tab)
- Reading the performance score (0-100, weighted calculation)
- The metrics section: finding FCP, LCP, TBT, CLS
- Color coding: green (good), orange (needs improvement), red (poor)
- Lab data vs. field data: what Lighthouse measures vs. real users
- PageSpeed Insights: seeing field data from real Chrome users
- The "Opportunities" and "Diagnostics" sections (preview for optimization learnpack)
- Why scores fluctuate: network conditions, CPU throttling, variability

Transitions: Gives students practical skills to measure their own applications. The assessment follows to verify understanding.

Key Principles:

- Lighthouse provides actionable lab measurements
- The score is useful but individual metrics matter more
- Field data (real users) is the ultimate truth

Examples:

- Running Lighthouse on a student project and identifying the LCP element
 - A site with 95 performance score but poor real-world experience (lab vs. field gap)
 - Finding that a 2MB hero image is causing poor LCP
-

Section 04: Assessment

04.0 Core Web Vitals Knowledge Check 🗨️ (~650 words)

Learning Objectives:

- Demonstrate understanding of Core Web Vitals metrics
- Apply measurement concepts to realistic scenarios

Assessment Format: Multiple choice (7 questions)

Questions:

1. Why should you measure frontend performance before trying to optimize it?
 - a) Because optimization tools require measurement data to work
 - b) Because without measurement, you might optimize the wrong thing and waste effort
 - c) Because browsers won't run optimized code without benchmarks
 - d) Because measurement is required by Google for SEO
2. A user visits your site and sees a blank white screen for 2 seconds, then text appears. After 4 more seconds, the large hero image loads. What are the approximate FCP and LCP values?
 - a) FCP: 2s, LCP: 2s
 - b) FCP: 6s, LCP: 6s
 - c) FCP: 2s, LCP: 6s
 - d) FCP: 4s, LCP: 6s
3. What does Largest Contentful Paint (LCP) measure?
 - a) The time until the largest JavaScript file finishes downloading
 - b) The time until the largest visible content element is rendered
 - c) The time until all content on the page has loaded
 - d) The time until the largest network request completes
4. A user clicks a button on your page, but nothing happens for 400 milliseconds. Which Core Web Vital metric relates to this problem?
 - a) Largest Contentful Paint (LCP)
 - b) First Contentful Paint (FCP)
 - c) Total Blocking Time (TBT) / First Input Delay (FID)
 - d) Cumulative Layout Shift (CLS)
5. What is the recommended threshold for a "good" LCP score?
 - a) Under 1.0 seconds
 - b) Under 2.5 seconds
 - c) Under 4.0 seconds
 - d) Under 5.0 seconds
6. Your Lighthouse report shows a performance score of 85, but your LCP is marked in red as "Poor." What should you focus on?
 - a) The overall score is good, so no action is needed
 - b) The individual LCP metric, because it directly affects user experience
 - c) Running the test again to get a better score
 - d) The FCP metric, since it loads before LCP
7. What is the main cause of high Total Blocking Time (TBT)?
 - a) Large images loading slowly
 - b) Long JavaScript tasks blocking the main thread
 - c) Slow network connections
 - d) Too many CSS files

Evaluation Criteria:

- 6-7 correct: Excellent understanding of Core Web Vitals
 - 4-5 correct: Good foundation, review specific metrics
 - Below 4: Review Section 02 content before proceeding
-

Section 05: Conclusion

05.0 From Measurement to Action (~400 words)

Content Outline:

- Recap: what students learned
 - Why measurement precedes optimization
 - The three Core Web Vitals: LCP, TBT (FID), CLS
 - How to read a Lighthouse report
 - Target thresholds for good performance
- Connection to what's next: optimization strategies
 - Students now know WHAT to measure
 - The next learnpack covers HOW to fix issues (code splitting, lazy loading, memoization)
- The ongoing practice of performance measurement
 - Measure before and after every change
 - Performance is not a one-time fix but continuous attention
- Resources for further exploration
 - web.dev/vitals (Google's Core Web Vitals documentation)
 - PageSpeed Insights for field data
 - Chrome DevTools Performance panel for deep analysis
- Encouragement: students now have the vocabulary and tools to discuss performance professionally

Note: This is conclusion only — no theory, exercises, or assessment.
